

Mundo de Wumpus

Visão abstraída do funcionamento

Quem faz o quê, e por onde a execução passa.

É orientado a objetos, de um sabor bem específico que vale nomear antes de tudo: **o núcleo é OO clássico** (objetos com estado privado e comportamento, encapsulamento real), enquanto **a camada gráfica é OO na estrutura e procedural nas ferramentas** — várias classes de desenho são caixas de função estática, sem estado. Isso não é acidente; onde importa aparece no fim.

1

O mapa mental: é um MVC, com o C repartido

<i>Papel</i>	<i>Onde mora</i>	<i>Quem é</i>
Model	nucleo	Mundo, AgenteInteligente + um vocabulário de tipos
View	grafico, console	duas apresentações intercambiáveis da mesma coisa
Controller do domínio	nucleo	Partida
Controller da interface	grafico	TelaDaPartida

A diferença para o MVC de livro é que **há dois “gerentes”, não um**, e cada um manda no seu lado da fronteira:

- Partida gerencia **o jogo**: de quem é a vez, o que acontece, quando acabou.
- TelaDaPartida gerencia **a janela**: o que aparece, onde, quando pedir o próximo passo.

Eles se falam por um único verbo, e é o ponto mais importante deste projeto inteiro:

```
TelaDaPartida — executarPasso() —> Partida
TelaDaPartida <— getAgente(), getMundo(), getRegistro()... — Partida
```

Uma seta de comando, várias de leitura. A tela manda uma única ordem — “avance um passo” — e todo o resto é consulta. É isso que faz o console e a janela serem trocáveis: os dois dão a mesma ordem e leem os mesmos getters, cada um desenhando do seu jeito.

2

As quatro famílias de classe, por responsabilidade

Há classes muito responsáveis e classes quase sem responsabilidade. São quatro grupos, e cada classe do projeto cai limpo em um deles.

Grupo A — As três que têm estado e regra (as “pesadas”)

Só três classes guardam **o estado do jogo**. Outras guardam estado — a tela tem relógios, progresso de animação, caches de layout — mas isso é *estado de apresentação*, e a distinção é justamente o que se quer ensinar: jogue fora a janela e o jogo continua íntegro.

- **Mundo** — é o *tabuleiro*. Sabe onde estão os poços, o Wumpus e o ouro, e quais casas já foram pisadas. É o **dono da verdade**. Não decide nada, não imprime nada: responde perguntas.
- **AgenteInteligente** — é o *explorador*. Guarda onde ele está, quanto pontuou, se tem ouro e flecha — e, mais importante, guarda **a cabeça dele**: quantas vezes visitou cada casa, de quais casas ele *suspeita*, e o caminho de volta que memorizou. Ele **nunca olha o mapa real**; só recebe percepções.
- **Partida** — é o *juiz*. É a única que vê os dois lados: ela pergunta ao mundo o que se sente, entrega isso ao agente, deixa o agente decidir, e então aplica as consequências do que ele fez. É a classe de “gerenciamento geral”.

A distinção entre Mundo e AgenteInteligente é o coração conceitual do trabalho: **o mundo sabe a verdade, o agente só tem crenças**. O Mundo sabe que há um poço em tal casa; o AgenteInteligente só sabe que sentiu brisa e que, portanto, *desconfia* das vizinhas. A tela desenha as duas coisas com origens diferentes — os elementos vêm do mundo, o véu vermelho de risco vem da cabeça do agente.

Grupo B — O vocabulário (as “leves”)

Cerca de dez tipos minúsculos, quase todos record ou enum, sem comportamento nenhum: Posicao, Direcao, Percepcoes, Decisao, EventoJogo, TipoEvento, Situacao, DisparoDeFlecha.

Eles existem para dar **nome próprio às coisas que trafegam entre as classes pesadas**. Em vez de passar `int linha, int coluna` soltos, passa-se uma `Posicao`. Em vez de devolver uma string pronta dizendo “andou para CIMA porque...”, o agente devolve uma `Decisao` — um objeto que a apresentação formata como quiser.

Dois merecem destaque porque carregam o fluxo:

- **EventoJogo** é o que substituiu os `System.out.println`. Toda vez que algo acontece, a `Partida` cria um `EventoJogo` e joga numa lista. O console formata como linha de texto; a janela desenha colorido. **Mesma informação, duas leituras.**
- **Situacao** é o semáforo do jogo: `EM_ANDAMENTO`, `VITORIA`, `MORTE`, `LIMITE_ATINGIDO`. Quase toda pergunta de “e agora?” acaba consultando ele.

Grupo C — As regiões da tela (uma classe por pedaço da janela)

Seis classes, cada uma dona de um retângulo: `BarraSuperior` (topo), `PainelTabuleiro` (o mapa), `PainelStatus` (as informações), `PainelRegistro` (o log rolante), `PainelRodape` (legenda), `CamadaResultado` (o cartão do fim).

Aqui está a diferença mais notável em relação a um MVC clássico: esses painéis **não herdam de uma tela-mãe**. Não existe classe abstrata nem interface comum. O que existe é uma **convenção** — os seis têm os mesmos três métodos, com os mesmos nomes:

<code>definirArea(...)</code>	"este é o seu retângulo, se organize dentro dele"
<code>desenharFormas(...)</code>	"desenhe seus fundos, caixas e figuras"
<code>desenharTextos(...)</code>	"agora escreva seus textos"

`TelaDaPartida` chama os seis em sequência, como uma orquestra. Aprendeu a ler um painel, leu os seis. A herança foi dispensada porque não há nada a herdar: não compartilham estado, só o formato — e nem o formato é idêntico, já que as assinaturas dos seis métodos divergem entre si. A convenção está nos nomes, não num tipo.

Por que desenhar é partido em dois métodos. Não é gosto: formas e texto usam desenhadores diferentes da `libGDX`, e alternar entre os dois a cada painel é caro. Por isso o quadro sai em duas varreduras completas — todas as formas, depois todos os textos — e por isso cada painel é partido ao meio. Juntar `desenharFormas` e `desenharTextos` “para simplificar” quebra o desempenho, não a organização.

Um dos seis tem peso extra: **`PainelTabuleiro` é o único lugar do projeto que traduz linha/coluna em coordenada de tela**, e ele inverte o eixo Y — a matriz conta linhas de cima para baixo, a `libGDX` conta Y de baixo para cima. Qualquer coisa que precise saber onde uma casa está na janela pergunta a ele em vez de recalcular. Se a fórmula estiver errada, há exatamente um arquivo para corrigir.

Grupo D — A caixa de ferramentas (sem estado)

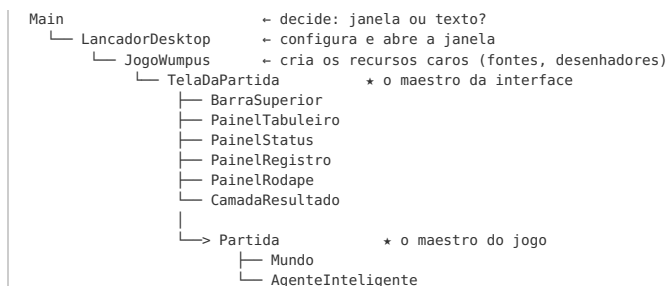
`Desenho`, `Icones`, `Paleta`, `Ativos`, `Efeitos`, `Area`.

Não são objetos do domínio, são **utensílios**. `Paleta` é a lista de cores. `Desenho` sabe fazer cantos arredondados e sombras. `Icones` sabe desenhar um agente, um poço, um Wumpus — tudo por geometria, sem arquivo de imagem. `Ativos` guarda as fontes. `Efeitos` é o único com um pinga de estado: a lista de tremores e clarões ainda acontecendo.

Qualquer painel pode chamar qualquer ferramenta. É a parte procedural mencionada no início: são funções, não objetos.

A cadeia de comando

Duas linhagens, que se tocam num ponto só:



Repare que a seta para Partida sai de TelaDaPartida, não de JogoWumpus. Faz sentido: quem precisa do jogo é quem o desenha e o comanda.

E repare no que **não** existe: nenhuma seta de volta. Partida não tem referência para tela nenhuma. Mundo não sabe que existe um AgenteInteligente. AgenteInteligente não sabe onde estão os poços. Cada um só conhece quem está abaixo dele.

4

O fluxo: dois laços que se encontram num método

Existem **dois laços rodando em ritmos diferentes**, e um deles cutuca o outro de vez em quando.

O laço rápido — 60 vezes por segundo

TelaDaPartida recebe da libGDX, a cada quadro, a pergunta “quanto tempo passou desde o último?”. Ela faz duas coisas: **atualiza e desenha**.

Esse laço não é escrito pelo projeto: TelaDaPartida estende ScreenAdapter da libGDX e preenche três métodos dela — render(delta) a cada quadro, resize() quando a janela muda de tamanho, hide() ao sair. **Ninguém no projeto chama nenhum dos três; a biblioteca é que chama**. É a inversão de controle: depois que a janela abre, quem dita o ritmo é o framework.

Desenhar é sempre a mesma rotina: percorre os seis painéis, cada um lê o estado atual da partida e pinta sua região. Isso acontece **60 vezes por segundo, aconteça o que acontecer** — mesmo com o jogo pausado, mesmo depois de terminado. A tela é um espelho contínuo do estado; ela não “recebe avisos” de que algo mudou.

O laço lento — uma vez a cada ~0,6 s

Dentro do “atualizar”, há um relógio que vai somando o tempo decorrido. Quando ele estoura o intervalo configurado, acontece a única coisa relevante do dia:

```
| partida.executarPasso()
```

E o relógio zera. É isso. É o equivalente ao **Timer do Swing**, feito com um acumulador em vez de uma thread — e é por isso que a janela nunca trava: ela não espera o agente, ela pergunta as horas.

As teclas de velocidade só mexem no tamanho desse intervalo. A pausa só impede o relógio de correr — quem decide isso é uma máquina de quatro estados na tela (PARADO, JOGANDO, PAUSADO, FIM), que governa se o acumulador avança. Nenhum desses controles toca no jogo.

O que acontece dentro de um passo

Partida.executarPasso() é uma receita fixa de seis etapas, sempre na mesma ordem — e essa ordem é a lição do trabalho:

1. **Perceber** — pergunta ao Mundo o que se sente nesta casa (brisa? fedor? brilho?)
2. **Deduzir** — entrega isso ao AgenteInteligente, que aumenta a suspeita sobre as vizinhas desconhecidas
3. **Reagir** — se sentiu fedor e ainda tem a flecha, atira na vizinha mais suspeita
4. **Mover** — o agente dá nota a cada vizinha (casa nova vale bônus, risco e repetição valem penalidade) e anda para a melhor
5. **Resolver** — a Partida olha o que há na casa onde ele parou e aplica: poço mata, Wumpus é briga, ouro é prêmio
6. **Julgar** — morreu? voltou ao início com o ouro? estourou o limite de movimentos?

Cada coisa que acontece nessas etapas vira um EventoJogo numa lista. **Dois listas, na verdade**: o histórico completo (que o painel de registro exhibe) e só os deste passo (que a tela usa para disparar o tremor, o clarão, a onda).

A animação: o truque que costura os dois laços

O agente, no modelo, **pula** de uma casa para a outra — não existe meio-caminho. Mas na tela ele desliza.

A costura é esta: a `Partida` guarda **de onde ele saiu** e **para onde ele foi** no último passo. A tela, que roda 60× mais rápido, sabe há quanto tempo o passo aconteceu, e desenha o agente num ponto intermediário entre as duas casas, proporcional a esse tempo. Quando o próximo passo chega, ele já está no destino e a conta recomeça.

— O modelo é discreto; a fluidez é ilusão da apresentação. Nenhuma regra sabe que existe animação.

5

Um detalhe que revela a arquitetura inteira: o reiniciar

Quando se aperta `R`, o que acontece é:

```
| partida = new Partida(...)    ← joga fora o jogo e faz outro
```

Um objeto `Partida` novo, com um `Mundo` novo e um `AgenteInteligente` novo. **Os seis painéis não são recriados.** Eles continuam os mesmos objetos, e no quadro seguinte simplesmente começam a espelhar um jogo diferente.

Isso só funciona porque os painéis **não guardam nada sobre o jogo** — eles guardam apenas o próprio retângulo na tela. Tudo que sabem, perguntam à `Partida` na hora de desenhar. É a prova prática de que a separação é real e não decorativa: dá para trocar o modelo por baixo da view sem a view perceber.

Com uma ressalva que vale para quem for criar um sétimo painel: o reiniciar também limpa duas coisas à mão — a fila de efeitos e o cartão de resultado, que guardam animações em curso. A regra exata, então, não é “a view não guarda nada”, e sim **a view não guarda nada que não saiba se reiniciar**. Painel novo com cache tem que entrar nessa lista.

6

Como a fronteira se sustenta

O enunciado exige que a parte gráfica não “cole” — não pode consultar as posições escondidas para decidir pelo agente. Três disciplinas garantem isso, e todas seguem o mesmo princípio: **quem desenha, lê; quem decide, não vê**.

- O agente recebe o `Mundo` como parâmetro, mas a única coisa que ele pergunta é se uma casa existe dentro dos limites; tudo o mais vem pela **porta de percepção**, que devolve apenas brisa, fedor e brilho. **Isso é convenção, não impossibilidade** — o método que revela o conteúdo de uma casa é público, e nada no compilador impede uma chamada distraída. É a regra que mais precisa ser olhada em revisão de código.
- O tabuleiro desenha o conteúdo de uma casa apenas se ela **já foi visitada** — ou se a partida acabou, e aí o mapa inteiro é revelado de uma vez.
- O véu de risco vermelho não vem do mapa: vem da **memória do agente**. Aquilo que se vê na tela é o que ele acha, não o que é.

E o pacote `nucleo` não importa uma linha de `libGDX`. Ele compila, roda e é testado sem que exista janela alguma — o modo console e os testes automatizados são a demonstração viva disso.

7

A rede de segurança

São 29 testes automatizados, **todos sobre `nucleo`, nenhum sobre `grafico`**. A consequência é operacional e vale saber no primeiro dia: mexer nas regras é protegido — se você quebrar alguma, a suíte acusa. Mexer no desenho não tem rede; verifica-se rodando e olhando.

```
| ./gradlew run      # janela gráfica (o padrão)
| ./gradlew console  # a mesma simulação em modo texto
| ./gradlew test     # os 29 testes das regras
```

No Windows, `gradlew.bat` no lugar de `./gradlew`. Três coisas que fazem perder uma tarde se ninguém avisar:

- **JDK 17 ou superior.** O projeto compila com *release* 17, então qualquer JDK a partir daí serve — não é preciso instalar o 17 especificamente.
- **Internet na primeira execução, e só nela.** O `gradlew` não é o Gradle: é um script que baixa o Gradle e, depois, a `libGDX` e as bibliotecas nativas. A primeira execução demora alguns minutos e *parece travada*; as seguintes são instantâneas e funcionam offline. Não instale Gradle nem `libGDX` à mão — o wrapper resolve.

- **No macOS, rodando pela IDE**, é preciso acrescentar `-XstartOnFirstThread` às opções da JVM, ou a janela não abre. Pelo `./gradlew run` isso já vem configurado.

8

Por onde começar a ler o código

O princípio que mais economiza tempo é contraintuitivo: **leia o modo console antes de qualquer coisa gráfica**. Ele é o mesmo jogo com a libGDX removida — na prática, a implementação de referência de “como ser cliente da *Partida*”. Quem entende as quarenta linhas dele já entende o que a janela faz; só falta descobrir como ela faz sem travar.

Antes de tudo, trinta segundos em `Main.java` só para ver a bifurcação. Depois, nesta ordem:

1. **console/ModoConsole.java** — o método `executar()`. O jogo inteiro cabe num `while`: cria uma *Partida*, chama `executarPasso()` em laço, imprime os eventos novos, para quando a situação encerra. Repare no que ele *não* faz: não conhece regra, não decide nada, não sabe onde estão os poços. É o contrato com o núcleo inteiro, em vinte linhas.
2. **nucleo/Partida.java** — o método `executarPasso()`. As seis etapas, sempre na mesma ordem. Confirme que este é o único método público que muda alguma coisa; todo o resto da classe é `getter`. Se você entender só um método do projeto, que seja este.
3. **nucleo/AgenteInteligente.java** — o cérebro. Procure três coisas: a fórmula da nota (casa nova dá bônus, risco e repetição penalizam); o fato de que observar só aumenta suspeita de casas *desconhecidas*; e o truque de truncar a lista do caminho quando o agente repisa uma casa — é isso, e só isso, que elimina as voltas em círculo no retorno. Repare que o Mundo chega como parâmetro, nunca como campo.
4. **grafico/TelaDaPartida.java** — agora sim o gráfico, e você já sabe o que ele quer. Procure onde foi parar o `while` do console: virou um acumulador de tempo. E repare que o desenho percorre os painéis duas vezes, uma para formas e outra para textos. Esses dois detalhes explicam o formato de toda a camada gráfica.
5. **grafico/PainelTabuleiro.java** — o único tradutor linha/coluna → pixel. Procure o `if` que decide se uma casa pode ser desenhada (visitada ou revelada): é a guarda que sustenta a regra do enunciado, e está numa linha só.

Somando: cerca de **250 linhas relevantes**. Na primeira leitura, ignore o Grupo D inteiro — são ferramentas de pintura, não contam nada sobre o funcionamento. Volte a elas só quando precisar desenhar algo.

9

Onde a arquitetura quebra sem ninguém perceber

O ponto cego é estrutural e vale dizer em voz alta: **os 29 testes cobrem apenas nucleo**. Qualquer coisa feita em `grafico` passa no `test` por construção — não porque está certa, mas porque ninguém olhou. Os quatro riscos abaixo passam em tudo sem um aviso sequer.

1. Importar libGDX dentro do núcleo

A libGDX está no classpath de compilação dos testes. Então uma linha tão inocente quanto `import com.badlogic.gdx.math.MathUtils`; dentro do agente — “só para usar o `clamp`” — compila, passa nos 29 testes, roda no console e roda na janela. Nada acusa. Mas a propriedade que sustenta o trabalho inteiro, *as regras existem sem janela*, morreu naquela linha.

É o risco número um porque é o mais tentador, o mais invisível e o mais difícil de reverter depois que se espalha. O único detector é um `grep`.

2. Guardar o Mundo como campo do agente

Hoje o Mundo chega ao agente por parâmetro de método, nunca como atributo. Isso não é desleixo: é a garantia. O agente pega o mundo emprestado durante uma chamada e o devolve — não consegue consultar o mapa fora da sua vez.

O refactor tentador é óbvio: “por que passar `mundo` em todo método? Guarda no construtor.” Comportamento idêntico, código mais curto, 29 testes verdes. E a garantia caiu de *emprestado por uma chamada* para *posse permanente*. É o exemplo perfeito da categoria: não remove nem adiciona classe, não muda comportamento, passa em tudo, e enfraquece a arquitetura.

3. Afrouxar a guarda de revelação no tabuleiro

O que impede o mapa de vaziar é uma condição só, que decide se a casa é conhecida. O painel já tem o Mundo em mãos e já consulta as percepções para desenhar os selos de brisa e fedor nas casas visitadas — legítimo, porque o agente já sentiu aquilo. **A distância entre o uso legítimo e o ilegítimo é um if**. Quem quiser “só mostrar uma dica” ou “deixar o poço meio transparente

para ficar bonito” descumpre o requisito central do enunciado, e nenhum teste existe nessa camada.

4. *Dois menores, de uma linha cada*

- Um `System.out.println` de depuração esquecido no núcleo. As mensagens viraram objetos `EventoJogo` justamente para que as regras nunca imprimissem nada; os testes não verificam a saída padrão.
- Recalcular na view um número que o modelo já sabe. A partir daí existem duas verdades sobre a pontuação, e elas divergem no primeiro ajuste de regra. Número novo é método novo no núcleo, nunca conta na tela.

O guarda-corpo, em três linhas. Cobre os dois riscos invisíveis e roda em segundos:

```
grep -rn "badlogic" src/main/java/wumpusworld/nucleo/ # deve voltar vazio
grep -rn "System.out" src/main/java/wumpusworld/nucleo/ # deve voltar vazio
./gradlew test && ./gradlew console
```

Os dois `grep` são, na prática, os testes que faltam na suíte. Como tarefa do Gradle que falha o build, a disciplina deixa de depender de alguém lembrar.

Resumindo em uma frase. Três classes guardam o jogo, uma delas arbitra; seis classes desenham pedaços de janela e uma delas rege; entre os dois mundos passa um único comando — “avance um passo” — e um punhado de perguntas.